SPOTIFY • BIGBASKET DATASET • DATA ANALYTICS



SPOTIFY ANALYSIS

DATA ANALYTICS

Exploratory Data Analysis with Python

27,555 PRODUCTS • 10 COLUMNS • 9 PYTHON STEPS

Better Data → Better Decisions → Smarter Retail

SPOTIFY | DATA ANALYTICS

TABLE OF CONTENT

What this project covers, step by step

01	About the Project	3
02	Objective	4
03	Tools & Graphs Used	5
04	Column Description	6
05	Data Overview	7
06	Step 1 - Load Dataset	8
07	Step 2 - First 12 Rows: head()	9
08	Step 3 - Description: describe()	10
09	Step 4 - Information: info()	11
10	Step 5 - Top & Least Products	12
11	Step 6 - Measuring a Discount	13
12	Step 7 - Missing Values	14
13	Step 8 - Outliers (filled with Mean)	15
14	Step 9 - Visualisations (4 pages)	16 - 19
15	Conclusion	20

▶ Project Flow

1 Title → **2** Objective → **3** Dataset Description → **4** Python Steps 1-9 → **5** Output Screenshots → **6** Conclusion

How each Python step is presented

- **Explanation** a short note in the same tone as the project reference.
- **Python code** a clean code block with line numbers and syntax colours.
- **Output screenshot** the real output generated by running the code on the dataset.
- **Key insight** what the output tells us about the products.

Better Data → Better Insights

ABOUT THE PROJECT

SPOTIFY PROJECT ANALYSIS

Online grocery • Prices • Ratings • Categories

This project performs an **Exploratory Data Analysis (EDA)** on the BigBasket product catalogue, presented in a Spotify-inspired green and black theme. The focus is on exploring, cleaning and visualising the data. No machine-learning model is used.

ABOUT THE DATASET

The file BigBasket_Products.csv holds 27,555 product listings. Each row is one product, described by 10 columns: name, category, sub-category, brand, sale price, market price, type, rating and description.

Source: BigBasket product catalogue (CSV supplied with the assignment).

WHAT IS EDA?

EDA means looking at data before any modelling: checking its structure, quality, distribution and relationships so that patterns and problems show up early.

Result: clean data and clear charts.

ASSESSMENT GOALS

- Gain familiarity with the dataset.
- Perform data exploration and visualisation.
- Identify patterns, trends and potential insights.
- Generate meaningful visualisations to communicate findings.

PROJECT TASKS

- **Task 1:** Data exploration
- **Task 2.1:** Univariate analysis
- **Task 2.2:** Bivariate analysis
- **Task 2.3:** Multivariate analysis
- **Task 3:** Geospatial (optional)

SUBMISSION GUIDELINES

- Organise the work in a Jupyter Notebook or similar.
- Explain and interpret every visualisation.
- Comment the code so it is easy to follow.
- Include graphs, charts and plots as support.

NOTES ON THE DATA

- There is no sales or quantity column, so "top and least sold" uses price and listing counts.
- Prices are in Indian rupees (Rs.); rating is on a 1 to 5 scale.
- Spotify is used only as the design theme; the data is BigBasket.
- Task 3 (maps) is skipped: the data has no latitude or longitude.

MORE THAN JUST A DATASET
IT’S A RETAIL STORY

SPOTIFY | DATA ANALYTICS

OBJECTIVE OF THE PROJECT

Why we analyse the product dataset

1	**Understand the Catalogue** Study what is sold: categories, sub-categories, brands and product types.
2	**Check Data Quality** Find missing values and duplicates, and treat them so results can be trusted.
3	**Explore Prices & Ratings** Measure the spread of sale price, market price and customer rating.
4	**Detect & Treat Outliers** Use the IQR method to find extreme values and replace them with the mean.
5	**Study Discounts** See how many products are sold below the market price and how this varies by category.
6	**Compare Categories** Compare categories by product count, average price and average rating.
7	**Visualise & Communicate** Use histograms, bar, donut, scatter, violin, heat-map and stacked charts to explain the findings.

For this Spotify Analysis Project

Objective: To analyse the product dataset and identify trends and patterns in categories, brands, sale price, market price, discount and rating, so that the distribution of the catalogue and the quality of the data are clearly understood.

Clean Data → Better Insights

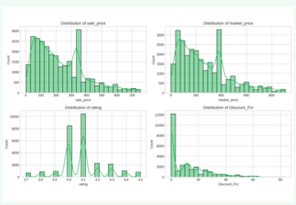
TOOLS & GRAPHS USED

Data tells the next story

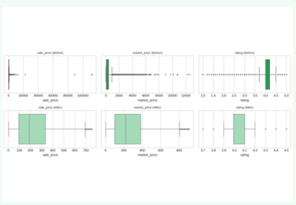
T Tools Used

Py	pd	np	plt	sns	Jup
Python	**Pandas**	**NumPy**	**Matplotlib**	**Seaborn**	**Jupyter / Colab**
Cleaning, analysis & coding	Data manipulation & summaries	Numerical operations	Basic & custom plots	Advanced statistical plots	Notebook environment

G Graphs Used



Histogram
Distribution of numbers
Step 9.1



Box Plot
Spread & outliers
Step 8



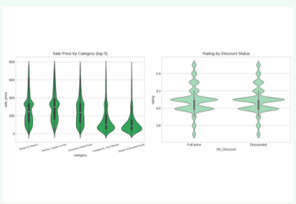
Bar Chart
Category counts
Step 9.2



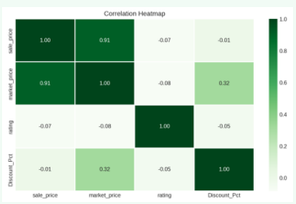
Donut Chart
Share of categories
Step 9.2



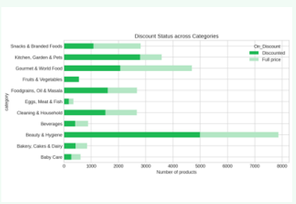
Scatter Plot
Two numeric variables
Step 9.3



Violin Plot
Number across categories
Step 9.3



Heatmap
Correlation matrix
Step 9.4



Stacked Bar
Category vs category
Step 9.4

All charts shown here are real: they were generated by running the Python code on BigBasket_Products.csv. See Steps 8 and 9 for the code and the interpretation.

Better Data → Better Decisions

SPOTIFY | DATASET

COLUMN DESCRIPTION

Understanding the 10 columns

#	Column	Data Type	Description
1	**index**	int64	Serial number (unique ID) of each product listing.
2	**product**	object	Name of the product as listed on BigBasket.
3	**category**	object	Top-level product category (11 groups).
4	**sub_category**	object	Sub-group inside a category (90 groups).
5	**brand**	object	Brand that makes or sells the product (2,313 brands).
6	**sale_price**	float64	Price at which the product is sold, in Rs.
7	**market_price**	float64	Printed market price (MRP) of the product, in Rs.
8	**type**	object	Specific product type (426 types).
9	**rating**	float64	Customer rating from 1 to 5 (blank when not yet rated).
10	**description**	object	Text description of the product.

IDENTITY
index, product

PRODUCT PROFILE
category, sub_category, brand, type

PRICE & FEEDBACK
sale_price, market_price, rating, description

Know Your Columns → Know Your Data

SPOTIFY | DATASET

DATA OVERVIEW

Key statistics of BigBasket_Products.csv

The product dataset contains **27,555 records** and **10 columns**. It gives information about each product: name, category, sub-category, brand, sale price, market price, type, rating and description.

Key Dataset Statistics

Total Records	Total Columns	Numeric Columns	Text Columns
27,555	10	4	6

Missing Cells	Duplicate Rows	Max Price (Rs.)	Median Sale Price (Rs.)
8,759	0	112,475	190.3

Out *Composition of the catalogue (raw data, before cleaning)*

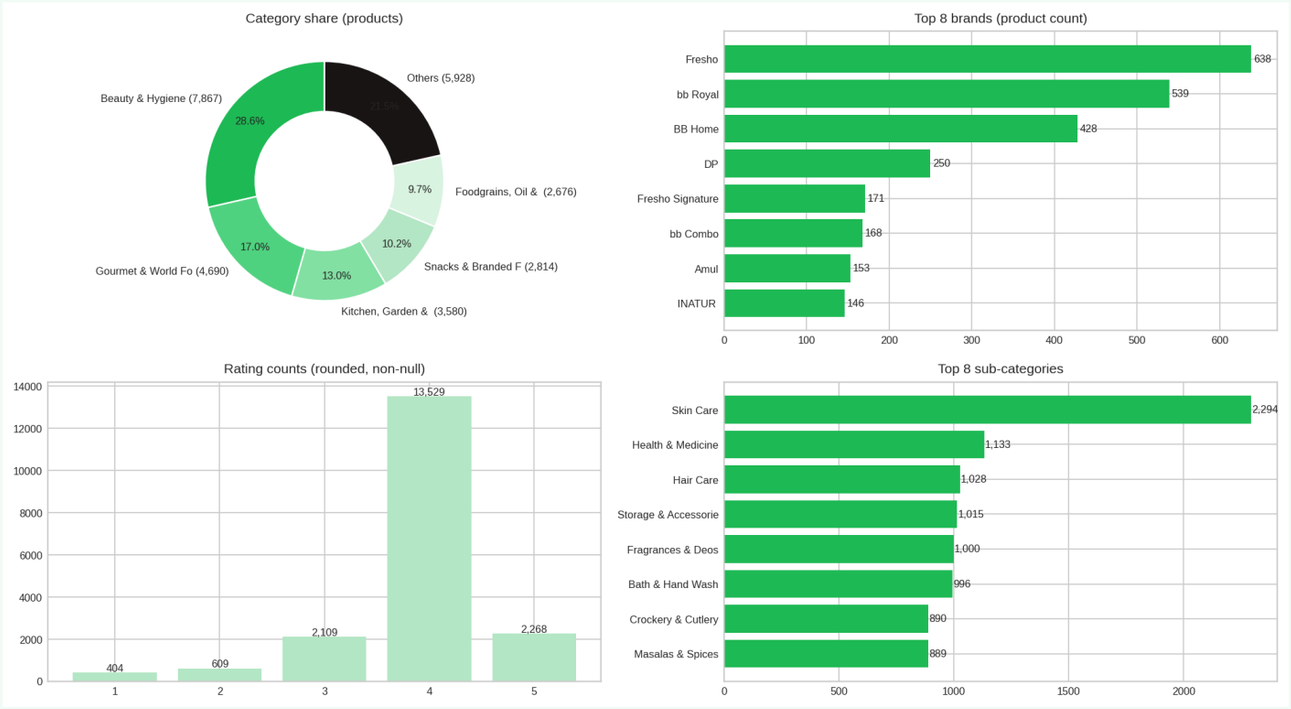


Figure 1: *Composition of the dataset: category share, top brands, ratings and sub-categories.*

WHAT STANDS OUT

- Beauty & Hygiene is the largest category with 7,867 products (28.6%), then Gourmet & World Food (4,690) and Kitchen, Garden & Pets (3,580).
- The catalogue spans 11 categories, 90 sub-categories and 2,313 brands.
- Fresho is the most listed brand (638 products), then bb Royal (539).
- Median rating is 4.1 out of 5, but only 18,919 products (69%) have a rating.

DATA QUALITY

- No duplicate rows: every product listing is different.
- 5 columns have missing values (8,759 cells in total).
- Rating is the weakest column: 8,636 products (31.3%) are not rated.
- Price columns contain extreme values (max Rs. 112,475 vs median Rs. 190) that need checking.

Clean Data → Better Insights

STEP 1 | PYTHON CODING

LOAD THE DATASET

Importing libraries & reading the CSV

Let’s load the product dataset with Python and Pandas. The goal is to import the libraries, set the chart theme and read the CSV file into a DataFrame called **df**.

1 Python Code

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Clean white theme + brand colour used for every chart
plt.style.use('seaborn-v0_8-whitegrid')
GREEN = '#1DB954'

# Load the BigBasket product dataset
df = pd.read_csv('/content/BigBasket_Products.csv')
print('Rows, Columns :', df.shape)
print('Column names:')
print(*df.columns, sep='\n')
```

What this code does?

- NumPy and Pandas are used for numbers and tables; Matplotlib and Seaborn for charts.
- plt.style.use('seaborn-v0_8-whitegrid') gives every chart a clean white theme with light grid lines.
- pd.read_csv() reads the CSV file into a DataFrame, and df.shape gives (rows, columns).
- The 10 column names are listed so we know what we are working with.

2 Output Screenshot

Out *Output of Step 1*

```
Rows, Columns : (27555, 10)
Column names:
index
product
category
sub_category
brand
sale_price
market_price
type
rating
description
```

Figure 2: *Shape and column names after loading the dataset.*

What it shows?

- The dataset has 27,555 rows and 10 columns.
- Columns are index, product, category, sub_category, brand, sale_price, market_price, type, rating and description.

Key Insight

Each row is one product listing. With 27,555 rows the dataset is large enough to show clear patterns in categories, brands and prices, and still light enough to run every step in seconds.

✔ Libraries imported	✔ Chart theme set	✔ CSV loaded	✔ Shape confirmed

From Data → To Stories

STEP 2 | PYTHON CODING

FIRST 12 ROWS - head()

Data exploration: understanding the structure

After loading the data, we look at the first 12 rows to understand the structure of the dataset. The table is shown in two parts so the text stays readable.

1 Python Code

```python
# First 12 rows of the dataset
df.head(12)
```

What this code does?

- df.head(12) returns the first 12 rows of the DataFrame.
- The first part shows the index with the product profile columns; the second part shows the index with price, type, rating and description.
- NaN marks a missing value.

2 Output Screenshot

Out *df.head(12): index with columns 2 to 5*

	index	product	category	sub_category	brand
0	1	Garlic Oil - Vegetarian Ca...	Beauty & Hygiene	Hair Care	Sri Sri Ayurveda
1	2	Water Bottle - Orange	Kitchen, Garden & Pets	Storage & Accessories	Mastercook
2	3	Brass Angle Deep - Plain, ...	Cleaning & Household	Pooja Needs	Trm
3	4	Cereal Flip Lid Container/...	Cleaning & Household	Bins & Bathroom Ware	Nakoda
4	5	Creme Soft Soap - For Hand...	Beauty & Hygiene	Bath & Hand Wash	Nivea
5	6	Germ - Removal Multipurpos...	Cleaning & Household	All Purpose Cleaners	Nature Protect
6	7	Multani Mati	Beauty & Hygiene	Skin Care	Satinance
7	8	Hand Sanitizer - 70% Alcoh...	Beauty & Hygiene	Bath & Hand Wash	Bionova
8	9	Biotin & Collagen Volumizi...	Beauty & Hygiene	Hair Care	StBotanica
9	10	Scrub Pad - Anti- Bacteria...	Cleaning & Household	Mops, Brushes & Scrubs	Scotch brite
10	11	Wheat Grass Powder - Raw	Gourmet & World Food	Cooking & Baking Needs	NUTRASHIL
11	12	Butter Cookies Gold Collec...	Gourmet & World Food	Chocolates & Biscuits	Sapphire

Figure 3: *head(12) output, index with the product profile columns.*

Out *df.head(12): index with columns 6 to 10*

	index	sale_price	market_price	type	rating	description
0	1	220.0	220.0	Hair Oil & Serum	4.1	This Product contains Garl...
1	2	180.0	180.0	Water & Fridge Bottles	2.3	Each product is microwave ...
2	3	119.0	250.0	Lamp & Lamp Oil	3.4	A perfect gift for all occ...
3	4	149.0	176.0	Laundry, Storage Baskets	3.7	Multipurpose container wit...
4	5	162.0	162.0	Bathing Bars & Soaps	4.4	Nivea Creme Soft Soap give...
5	6	169.0	199.0	Disinfectant Spray & Cleaners	3.3	Stay protected from contam...
6	7	58.0	58.0	Face Care	3.6	Satinance multani matti is...
7	8	250.0	250.0	Hand Wash & Sanitizers	4.0	70%Alcohol based is gentle...
8	9	1098.0	1098.0	Shampoo & Conditioner	3.5	An exclusive blend with Vi...
9	10	20.0	20.0	Utensil Scrub-Pad, Glove	4.3	Scotch Brite Anti- Bacteri...
10	11	261.0	290.0	Flours & Pre-Mixes	4.0	Wheatgrass is a superfood ...
11	12	600.0	600.0	Luxury Chocolates, Gifts	2.2	Enjoy a tin full of delici...

Figure 4: *head(12) output, index with price, type, rating and description columns.*

What it shows?

- Each row is one product; index is a running serial number.
- No NaN appears in these 12 rows, but the full file has 8,759 missing cells (found in Step 7).
- sale_price equals market_price for 8 of the 12 products, meaning no discount on them.
- Prices are in Rs.; the description column holds long text (shortened here).

Key Insight

The first rows already show mixed data types (numbers, text, long descriptions) and many products sold at full price, so we must check data quality before any analysis.

✔ Structure understood	✔ Types are mixed	✔ Long text seen	✔ Discounts seen

Clean Data → Better Insights

STEP 3 | PYTHON CODING

DATA DESCRIPTION - describe()

Statistical summary of numeric columns

The describe() function gives a quick statistical summary (count, mean, standard deviation, minimum, quartiles and maximum) for every numeric column.

1 Python Code

```
# Statistical summary of the numeric columns
df.describe()
```

What this code does?

- describe() works on the numeric columns only: index, sale_price, market_price and rating.
- count shows how many values are present; mean and std show the centre and spread.
- 25%, 50% (median) and 75% show how the values are spread.

2 Output Screenshot

Out *df.describe()*

	index	sale_price	market_price	rating
count	27555.00	27549.00	27555.00	18919.00
mean	13778.00	334.65	382.06	3.94
std	7954.59	1202.10	581.73	0.74
min	1.00	2.45	3.00	1.00
25%	6889.50	95.00	100.00	3.70
50%	13778.00	190.32	220.00	4.10
75%	20666.50	359.00	425.00	4.30
max	27555.00	112475.00	12500.00	5.00

Figure 5: *Statistical summary of the numeric columns (before cleaning).*

3 Mean vs Median

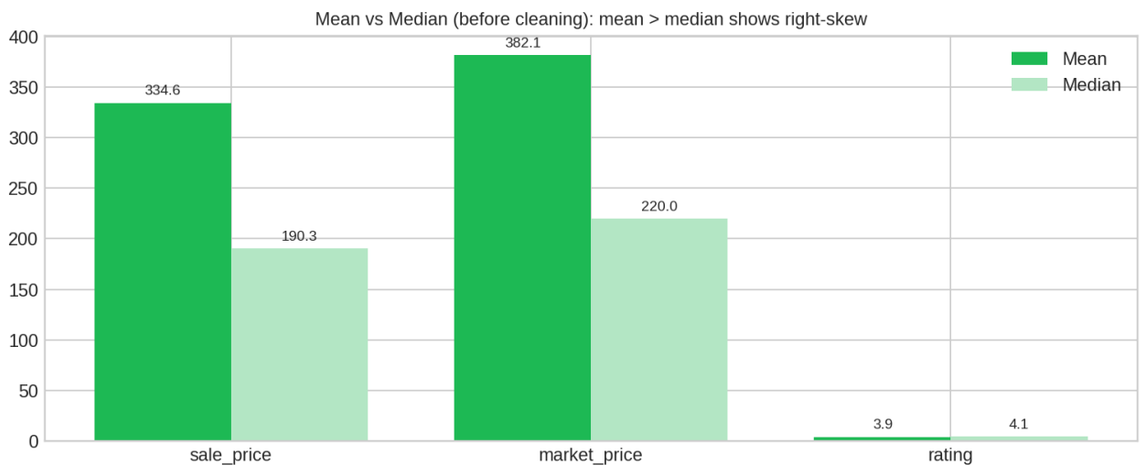


Figure 6: *A mean far above the median signals skewed data with large extreme values.*

What it shows?

- Count: sale_price 27,549 and rating 18,919 out of 27,555 rows, so they have gaps.
- sale_price: mean 334.6 but median 190.3, with a maximum of 112,475.
- market_price: mean 382.1, median 220.0, maximum 12,500.
- rating: mean 3.94, median 4.1, range 1.0 to 5.0.

Key Insight

Mean above median in both price columns means the data is right-skewed. A few very expensive products pull the averages up, so outliers must be handled.

✔ Skew detected	✔ Gaps in 2 columns	✔ Outliers likely	✔ Median rating is 4.1

Numbers → Meaning

STEP 4 | PYTHON CODING

FIND INFORMATION - info()

Data types, non-null counts & memory

The info() function shows the number of rows, every column with its non-null count and data type, and the memory used by the DataFrame.

1 Python Code

```python
# Structure, data types and non-null counts
df.info()
```

What this code does?

- info() lists each column with its non-null count and Dtype.
- A non-null count lower than the number of rows means the column has missing values.
- Memory usage tells us the DataFrame is small enough for a normal laptop.

2 Output Screenshot

Out *df.info()*

```
<class 'pandas.DataFrame'>
RangeIndex: 27555 entries, 0 to 27554
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   index         27555 non-null  int64
 1   product       27554 non-null  str
 2   category      27555 non-null  str
 3   sub_category  27555 non-null  str
 4   brand         27554 non-null  str
 5   sale_price    27549 non-null  float64
 6   market_price  27555 non-null  float64
 7   type          27555 non-null  str
 8   rating        18919 non-null  float64
 9   description   27440 non-null  str
dtypes: float64(3), int64(1), str(6)
memory usage: 2.1 MB
```

Figure 7: *info() output: 27,555 entries, 10 columns.*

3 Data Completeness

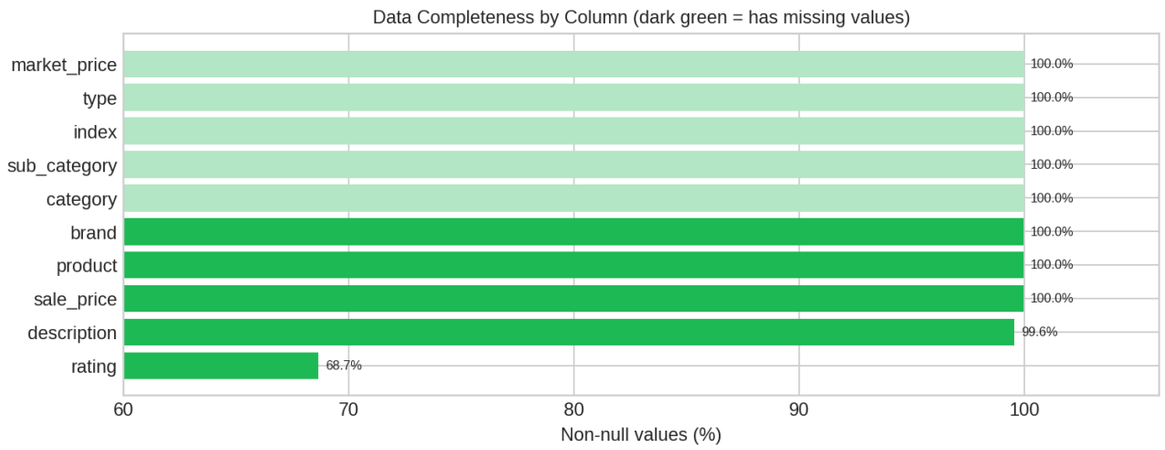


Figure 8: *Share of non-null values in each column. Dark-green bars have missing values.*

What it shows?

- 27,555 entries and 10 columns: 1 integer, 3 float and 6 text columns.
- Complete columns: index, category, sub_category, market_price and type.
- Gaps: rating 18,919, description 27,440, sale_price 27,549, product and brand 27,554 non-null.
- Newer pandas versions label text columns as str; older versions show object.

Key Insight

Rating has the most gaps (8,636 missing, about 31%). Because ratings reflect customer satisfaction, how we fill them matters.

✔ Types checked	✔ 5 columns incomplete	✔ Rating weakest	✔ Ready for cleaning

Better Data → Better Insights

STEP 5 | PYTHON CODING

TOP & LEAST PRODUCTS

Highest and lowest priced products, and category strength

The dataset has no sales quantity, so “top and least sold” uses proxies: the highest and lowest priced products, and the category with the most listings.

1 Python Code

```python
# Category segments: product count, average price and average rating
cat_summary = (df.groupby('category')
                 .agg(Products=('product', 'count'), Avg_Price=('sale_price', 'mean'),
                      Avg_Rating=('rating', 'mean'))
                 .sort_values('Products', ascending=False).round(1))
print(cat_summary)

# Top 10 and least 10 products by sale price (price in Rs.)
cols = ['product', 'brand', 'sale_price']
top10 = df.nlargest(10, 'sale_price')[cols].assign(product=lambda d: d['product'].str[:26])
least10 = df.nsmallest(10, 'sale_price')[cols].assign(product=lambda d: d['product'].str[:26])
display(top10)
display(least10)
```

What this code does?

- groupby() gives the product count, average price and average rating of each category.
- nlargest() and nsmallest() pick the 10 highest and 10 lowest priced products (names shortened to 26 characters).

2 Output Screenshot

Out *display(top10)*

	product	brand	sale_price
1249	Beard Kit	Uncle Tony	112475.00
248	4mm Aluminium Induction Ba	HAZEL	111649.00
436	Balloon - Polka Dot, 12 In	B Vishal	88899.00
288	Arrabbiata Tomato Pasta Sa	Montanini	22325.00
25301	Bravura Clipper	Wahl	12500.00
21761	Pet Food - N&D Team Breede	Farmina	10090.00
12669	Epilator SE9-9961 Legs-Bod	Braun	8184.44
23082	Gas Stove-4 Burner Royale	Prestige	7999.00
2781	Extra Virgin Olive Oil	Abbies	7299.00
25797	4 Burner Gas Stove - Marve	Prestige	7270.00

Out *display(least10)*

	product	brand	sale_price
26976	Curry Leaves	Fresho	2.45
21312	Serum	Livon	3.00
2761	Orbit Sugar-Free Chewing G	Wrigleys	5.00
2978	Sugar Free Chewing Gum - M	Orbit	5.00
3445	Marie Light Biscuits - Act	Sunfeast	5.00
6014	Good Day Butter Cookies	Britannia	5.00
9971	Tomato - Local, Organicall	Fresho	5.00
11306	Happy Happy Choco-Chip Coo	Parle	5.00
11435	Exam Standard Scale	Camlin	5.00
14184	Tomato - Local, Organicall	Fresho	5.00

Out *print(cat_summary)*

category	Products	Avg_Price	Avg_Rating
Beauty & Hygiene	7867	432.8	3.9
Gourmet & World Food	4690	324.5	4.0
Kitchen, Garden & Pets	3580	538.7	3.7
Snacks & Branded Foods	2814	129.6	4.0
Foodgrains, Oil & Masala	2676	193.2	4.1
Cleaning & Household	2675	259.5	4.0
Beverages	884	239.8	4.1
Bakery, Cakes & Dairy	851	142.8	3.9
Baby Care	610	534.9	4.0
Fruits & Vegetables	557	50.9	NaN
Eggs, Meat & Fish	350	288.9	NaN

Out *Top 10 vs Least 10 products (chart)*

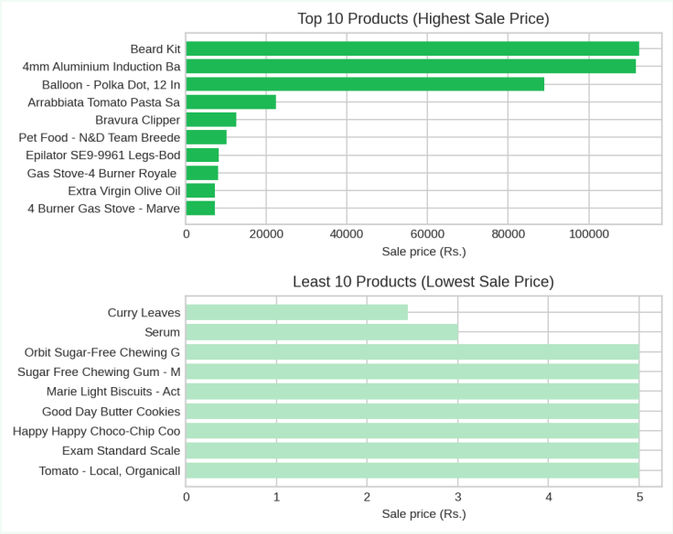


Figure 9: *Top 10 and least 10 products by sale price, category summary and the price chart (Rs.).*

What it shows?

- Beauty & Hygiene has the most listings: 7,867 products (28.6% of the catalogue).
- Highest average price: Kitchen, Garden & Pets (Rs. 538.7); lowest: Fruits & Vegetables (Rs. 50.9).
- Top priced item: Beard Kit at Rs. 112,475; then 4mm Aluminium Induction Ba (Rs. 111,649).
- Lowest priced items cost only Rs. 2.45.

Key Insight

Beauty & Hygiene leads the catalogue by count. Prices vary enormously: the dearest item costs about 45,908 times the cheapest. The top 3 prices (over Rs. 88,000) look like data-entry extremes and will be treated in Step 8.

✔ Top & least found	✔ Category ranked	✔ Prices compared	✔ Extremes spotted

Top & Least → Clear Picture

STEP 6 | PYTHON CODING

MEASURING A DISCOUNT

10% discount scenario on one category

To measure a discount on a certain item, we treat one category as the “item”: an extra 10% discount on the sale price of every Beauty & Hygiene product. We then compare the totals before and after the discount.

1 Python Code

```python
# Scenario: extra 10% discount on the sale price of every 'Beauty & Hygiene' item
discount_pct = 10
mask = df['category'] == 'Beauty & Hygiene'

df['Discounted_Price'] = np.where(mask, df['sale_price'] * (1 - discount_pct / 100), df['sale_price'])

before = df.loc[mask, 'sale_price'].sum()
after = df.loc[mask, 'Discounted_Price'].sum()
print('Beauty & Hygiene items :', mask.sum())
print('Total before discount  :', round(before, 1))
print('Total after discount   :', round(after, 1))
print('Discount given         :', round(before - after, 1), f'({discount_pct}%)')
print('Avg discount per item  :', round((before - after) / mask.sum(), 2))
mrp = (df['market_price'] - df['sale_price']) / df['market_price'] * 100
print('Existing MRP discount  :', round(mrp[mask].mean(), 1), '% (market vs sale price)')
```

What this code does?

- np.where() applies the discount only where the mask (category = Beauty & Hygiene) is True.
- The new column Discounted_Price keeps every other category unchanged.
- The totals before and after give the saving; the existing market-price discount is also printed for comparison.

2 Output Screenshot

Out *Discount calculation*

```
Beauty & Hygiene items : 7867
Total before discount  : 3404749.2
Total after discount   : 3064274.3
Discount given         : 340474.9 (10%)
Avg discount per item  : 43.28
Existing MRP discount  : 11.8 % (market vs sale price)
```

Figure 10: *Totals before and after the 10% Beauty & Hygiene discount.*

3 Before vs After Chart

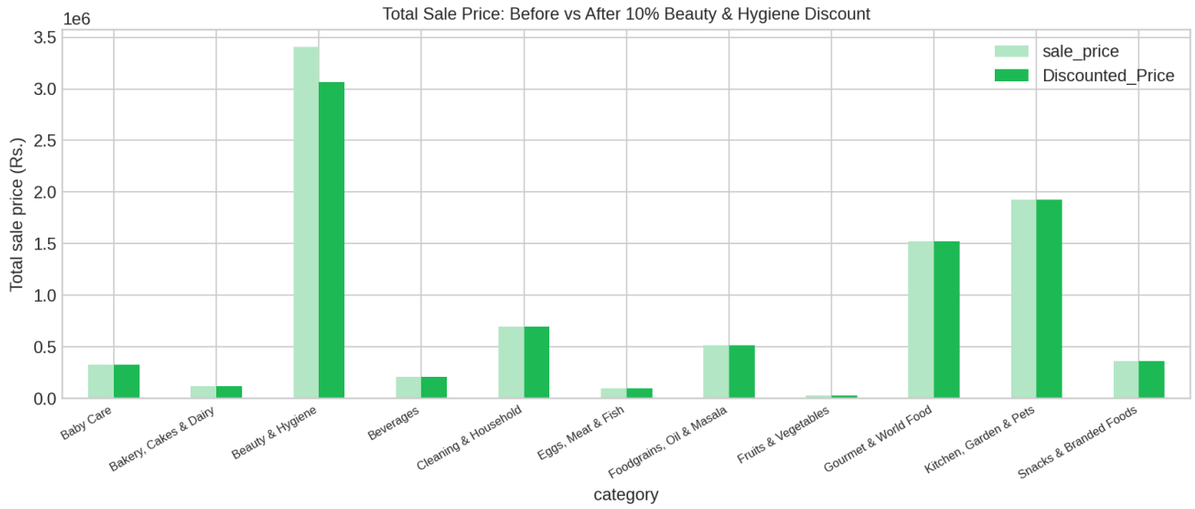


Figure 11: *Only the Beauty & Hygiene bar changes after the discount.*

What it shows?

- Beauty & Hygiene has 7,867 products with a total sale price of Rs. 3,404,749.2.
- After a 10% discount the total falls to Rs. 3,064,274.3, a saving of Rs. 340,474.9.
- The average discount is Rs. 43.28 per product.
- These products already carry about 11.8% discount versus the market price; all other categories stay the same.

Key Insight

The discount rate is an assumption for this scenario; change discount_pct or the category in the mask to test any other offer.

✔ Category selected	✔ 10% applied	✔ Savings measured	✔ Chart compared

Measure → Compare → Decide

STEP 7 | PYTHON CODING

MISSING VALUES

Finding gaps and handling them

Next we find the missing values in every column, view them in a chart, and then handle them so the later analysis is reliable.

1 Python Code

```python
# Count and percentage of missing values in each column
missing = df.isnull().sum()
miss_df = pd.DataFrame({'Missing': missing, 'Percent': (missing / len(df) * 100).round(2)})
print(miss_df)

# Handle: median for numbers, mode for text columns
for col in ['sale_price', 'rating']:
    df[col] = df[col].fillna(df[col].median())
for col in ['product', 'brand']:
    df[col] = df[col].fillna(df[col].mode()[0])
df['description'] = df['description'].fillna('Not available')
df['Discounted_Price'] = np.where(mask, df['sale_price'] * (1 - discount_pct / 100), df['sale_price'])
print('Missing values left :', df.isnull().sum().sum())
```

What this code does?

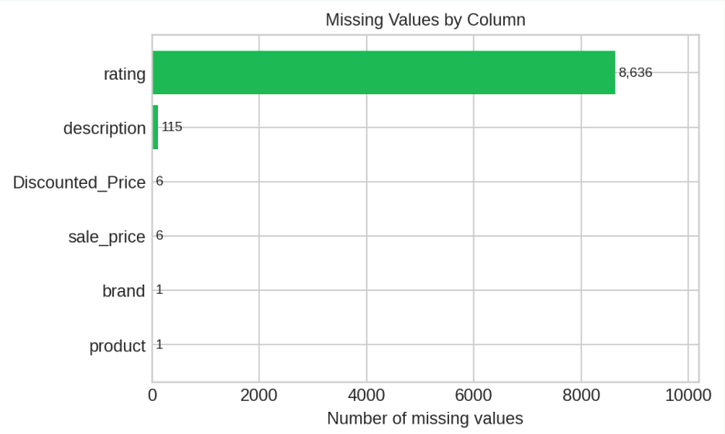
- isnull().sum() counts the missing cells in each column; dividing by the number of rows gives the percentage.
- Numbers (sale_price, rating) are filled with the median; text (product, brand) with the mode.
- Missing descriptions are filled with “Not available”, and Discounted_Price is recalculated.

2 Output Screenshot

Out *Missing values per column*

	Missing	Percent
index	0	0.00
product	1	0.00
category	0	0.00
sub_category	0	0.00
brand	1	0.00
sale_price	6	0.02
market_price	0	0.00
type	0	0.00
rating	8636	31.34
description	115	0.42
Discounted_Price	6	0.02

Out *Missing values by column*



Out *After handling*

```
Missing values left : 0
```

Figure 12: *Missing-value table, chart and the count after handling.*

What it shows?

- 8,759 cells are missing in 5 columns.
- Rating 8,636 (31.3%), description 115 (0.42%), sale_price 6, product 1 and brand 1.
- Discounted_Price also has 6 gaps because it is built from sale_price.
- After handling, 0 missing values remain.

Key Insight

The median is used for numbers because price and rating columns are skewed, and the mode keeps text columns valid (a real brand or product name). Ratings are 31% empty, so the median fill makes many products share one value.

✔ Gaps found	✔ Chart plotted	✔ Mode & median used	✔ 0 missing left

No Gaps → Trustworthy Data

STEP 8 | PYTHON CODING

OUTLIERS - IQR METHOD

Detect outliers and fill them with the mean

We find outliers column by column with the IQR rule (values below Q1 - 1.5×IQR or above Q3 + 1.5×IQR) and fill them with the column mean.

1 Python Code

```python
# IQR method: find outliers column-wise and fill them with the column mean
df_before = df.copy()                       # keep a copy for before/after plots
num_cols = ['sale_price', 'market_price', 'rating']
df[num_cols] = df[num_cols].astype(float)   # allow decimal mean values
report = {}

for col in num_cols:
    Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
    IQR = Q3 - Q1
    low, high = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    is_out = (df[col] < low) | (df[col] > high)
    report[col] = {'Lower Bound': round(low, 1), 'Upper Bound': round(high, 1),
                   'Outliers': int(is_out.sum()), 'Mean Used': round(df[col].mean(), 1)}
    df.loc[is_out, col] = df[col].mean()    # fill outliers with mean

out_report = pd.DataFrame(report).T.astype({'Outliers': int})
print(out_report)
```

What this code does?

- A copy of the data (df_before) is kept so we can draw before and after plots.
- For each column the IQR rule gives a lower and upper bound; values outside are outliers.
- Outliers are replaced with the column mean, so all rows are kept.

2 Output Screenshot

Out *Outlier report*

	Lower Bound	Upper Bound	Outliers	Mean Used
sale_price	-301.0	755.0	2205	334.6
market_price	-387.5	912.5	2147	382.1
rating	3.7	4.5	6372	4.0

Figure 13: *IQR bounds and number of outliers found in each column.*

3 Box Plots: Before vs After

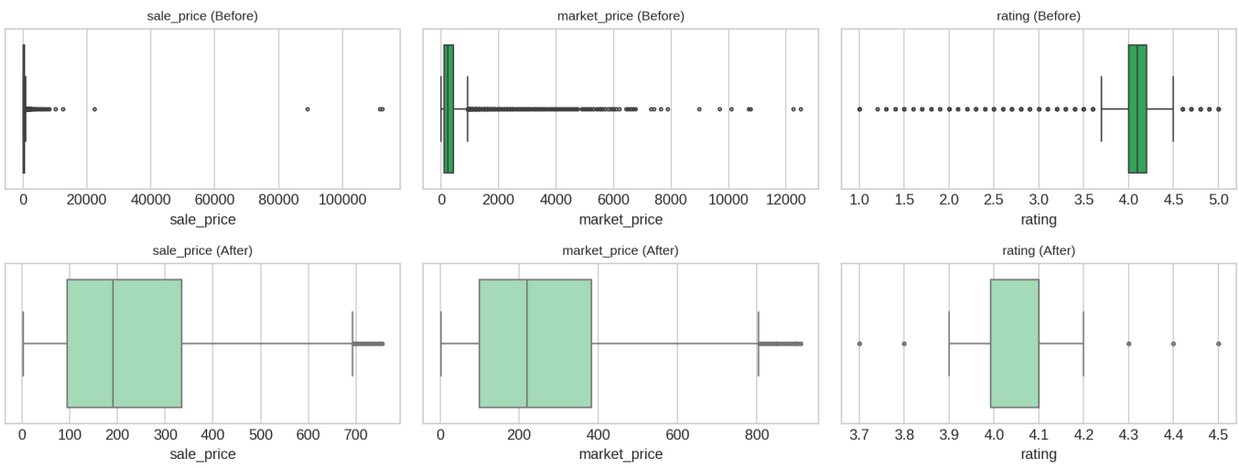


Figure 14: *Top row: before treatment. Bottom row: after outliers are replaced with the mean.*

What it shows?

- 10,724 outliers found: sale_price 2,205, market_price 2,147, rating 6,372.
- The maximum sale price of Rs. 112,475 was far above the upper bound of 755.
- After treatment, the whiskers are far shorter and the boxes are readable.
- index is excluded because it is only a serial number, not a measurement.

Key Insight

A few products cost up to about 591 times the median price (Rs. 112,475 vs Rs. 190). Replacing them with the mean keeps all rows but stops them from dominating charts. Rating shows the most flags because the median fill in Step 7 narrowed its IQR; treating each column on its own also changes some sale-to-market gaps.

✔ Bounds computed	✔ Outliers filled	✔ Rows kept	✔ Plots compared

Outliers → Under Control

STEP 9 | VISUALISATION

UNIVARIATE: HISTOGRAMS

Distribution of the numeric columns

Step 9 creates the visualisations on the cleaned data. First, the univariate analysis of the numeric columns with histograms and a KDE curve. A new column Discount_Pct (market price vs sale price) is added.

1 Python Code

```
# 9.1 Univariate analysis: histograms with KDE curve
df['Discount_Pct'] = ((df['market_price'] - df['sale_price']) / df['market_price'] * 100).clip(lower=0).round(1)
cols = ['sale_price', 'market_price', 'rating', 'Discount_Pct']
fig, ax = plt.subplots(2, 2, figsize=(12, 7))
for a, col in zip(ax.ravel(), cols):
    sns.histplot(df[col], kde=True, color=GREEN, ax=a, bins=25)
    a.set_title('Distribution of ' + col)
plt.tight_layout()
plt.show()
```

What this code does?

- Discount_Pct = (market_price - sale_price) / market_price × 100, never below 0.
- sns.histplot() draws the histogram; kde=True adds the smooth KDE curve.
- subplots(2, 2) places the four charts in one grid.

2 Output Screenshot

Out *Output chart*

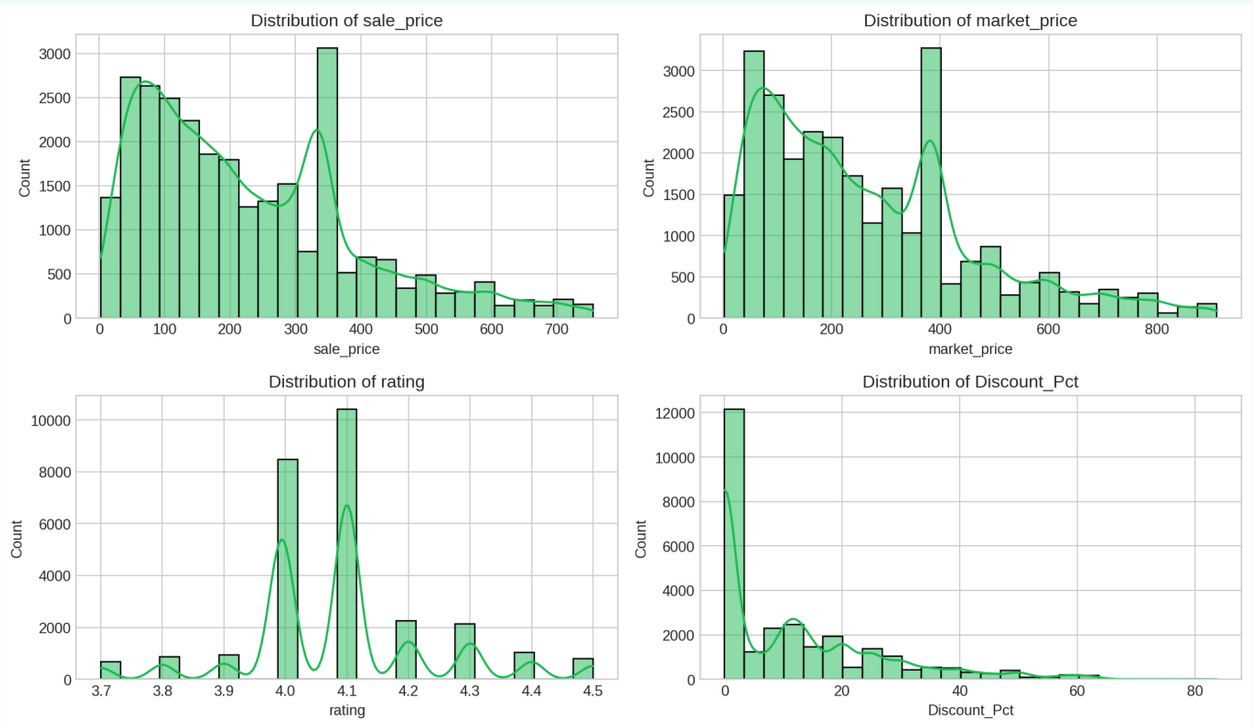


Figure 15: *Histograms of sale_price, market_price, rating and Discount_Pct.*

What it shows?

- sale_price: the middle 50% of products cost between Rs. 95 and Rs. 335, with a long tail to the right.
- market_price: the middle 50% are between Rs. 100 and Rs. 382.
- rating: values are packed around the typical score of 4.1, mostly between 4.0 and 4.1.
- Discount_Pct has a large spike at 0: 11,673 of 27,555 products (42.4%) are sold at full price.

Key Insight

Most products are everyday grocery and household items at modest prices; a large group is sold without any discount and most ratings sit between 3.7 and 4.5.

✔ Prices skewed right	✔ Rating clustered	✔ Zero-discount spike	✔ Distributions clear

Distribution → Understanding

UNIVARIATE: BAR & DONUT

Composition of categorical variables

Now the categorical variables: bar charts show the counts and donut charts show the composition.

1 Python Code

```python
# 9.2 Categorical variables: bar charts and donut charts
fig, ax = plt.subplots(1, 3, figsize=(13, 4.6))
for a, col, n in zip(ax, ['category', 'sub_category', 'brand'], [11, 10, 10]):
    vc = df[col].value_counts().head(n)[::-1]
    a.barh(vc.index.str[:20], vc.values, color=GREEN)
    a.set_title('Top ' + col + ' count')
plt.tight_layout()
plt.show()

df['On_Discount'] = np.where(df['Discount_Pct'] > 0, 'Discounted', 'Full price')
df['Rating_Band'] = np.where(df['rating'] >= 4, '4.0 and above', 'Below 4.0')
fig, ax = plt.subplots(1, 2, figsize=(12, 4.6))
for a, col in zip(ax, ['On_Discount', 'Rating_Band']):
    vc = df[col].value_counts()
    a.pie(vc.values, labels=vc.index, autopct='%1.1f%%', startangle=90,
          colors=['#1DB954', '#B3E6C4'], wedgeprops=dict(width=0.45, edgecolor='white'))
    a.set_title(col + ' Share')
plt.tight_layout()
plt.show()
```

What this code does?

- value_counts().head(n) gives the most frequent categories, sub-categories and brands.
- barh() draws horizontal bars so long names stay readable.
- Two helper columns (On_Discount, Rating_Band) are created for the donut charts.

2 Output Screenshot

Out *Output chart: bar charts*

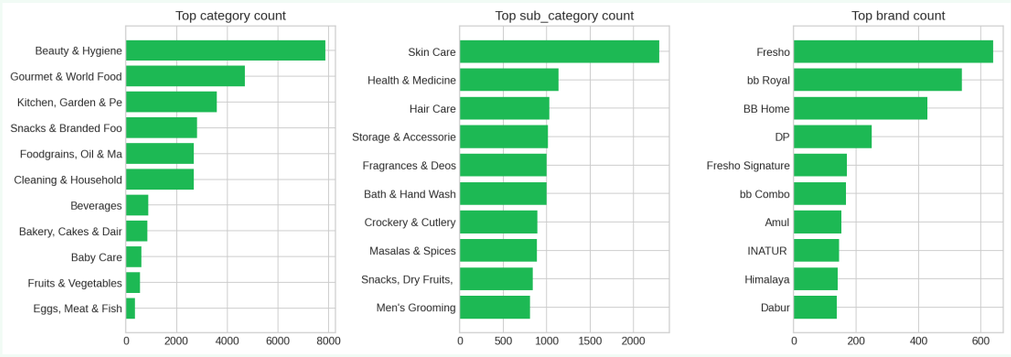


Figure 16: *Bar charts: category, sub_category and brand counts.*

Out *Output chart: donut charts*



Figure 17: *Donut charts: On_Discount and Rating_Band.*

What it shows?

- Beauty & Hygiene has 7,867 products (28.6%); Gourmet & World Food 4,690; Kitchen, Garden & Pets 3,580.
- Most listed brands: Fresho (638), bb Royal (539) and BB Home (428).
- 57.6% of products are sold below the market price; 42.4% at full price.
- 67.9% of products have a rating of 4.0 or higher.

Key Insight

The typical product is a beauty, gourmet or home item from a well-stocked brand, usually discounted and rated 4.0 or more.

✔ Counts compared	✔ Shares measured	✔ Top brands seen	✔ Donuts plotted

Categories → Clear Shares

STEP 9 | VISUALISATION

BIVARIATE ANALYSIS

Scatter plot and violin plots

Bivariate analysis checks how two variables relate: a scatter plot for two numeric columns and violin plots for a number across categories.

1 Python Code

```python
# 9.3 Bivariate analysis: scatter plot and violin plot
plt.figure(figsize=(10, 4.6))
sns.scatterplot(data=df, x='market_price', y='sale_price', hue='On_Discount',
                palette=[GREEN, '#191414'], alpha=0.5, s=12)
plt.title('Market Price vs Sale Price')
plt.tight_layout()
plt.show()

top5 = df['category'].value_counts().head(5).index
fig, ax = plt.subplots(1, 2, figsize=(12, 4.6))
sns.violinplot(data=df[df['category'].isin(top5)], x='category', y='sale_price', color=GREEN, ax=ax[0])
ax[0].set_title('Sale Price by Category (top 5)')
ax[0].tick_params(axis='x', rotation=15, labelsize=7)
sns.violinplot(data=df, x='On_Discount', y='rating', color='#9BE3B4', ax=ax[1])
ax[1].set_title('Rating by Discount Status')
plt.tight_layout()
plt.show()
```

What this code does?

- scatterplot() compares market_price (x) with sale_price (y) and colours the points by discount status.
- violinplot() shows the full spread of sale_price inside the 5 biggest categories.
- A second violin compares ratings of discounted and full-price products.

2 Output Screenshot

Out *Output chart: scatter plot*



Figure 18: *Scatter plot: market_price vs sale_price, coloured by discount status.*

Out *Output chart: violin plots*

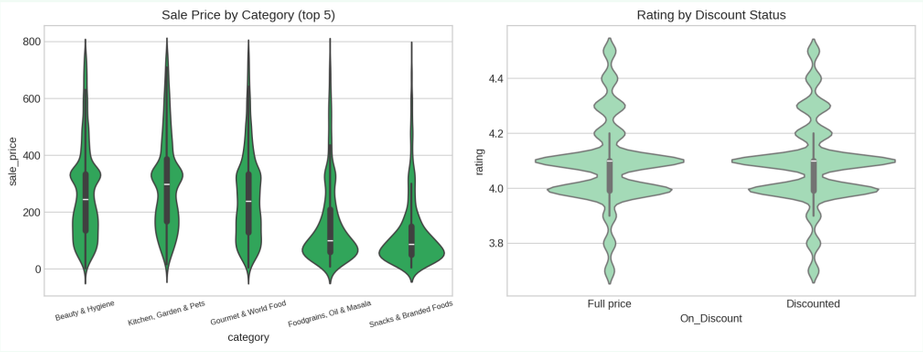


Figure 19: *Violin plots: Sale Price by Category and Rating by Discount Status.*

What it shows?

- Sale price rises almost in a straight line with market price (correlation 0.91).
- Average sale price: Kitchen, Garden & Pets Rs. 299, Beauty & Hygiene Rs. 266, Gourmet Rs. 252, Foodgrains Rs. 159, Snacks Rs. 125.
- Average rating is 4.09 for discounted and 4.09 for full-price products.

Key Insight

Market price almost fully decides the sale price, while the wide spread inside each category shows that category alone does not set the price. Discounts do not change how customers rate a product.

✔ Linear link found	✔ Categories compared	✔ Ratings compared	✔ Plots read

Two Variables → One Story

STEP 9 | VISUALISATION

MULTIVARIATE ANALYSIS

Correlation heatmap and stacked bar chart

Finally the multivariate analysis: a correlation heatmap for the numeric columns and a stacked bar chart of discount status across categories.

1 Python Code

```python
# 9.4 Multivariate analysis: correlation heatmap and stacked bar chart
plt.figure(figsize=(8, 5))
corr = df[['sale_price', 'market_price', 'rating', 'Discount_Pct']].corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='Greens', linewidths=1, linecolor='white')
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()

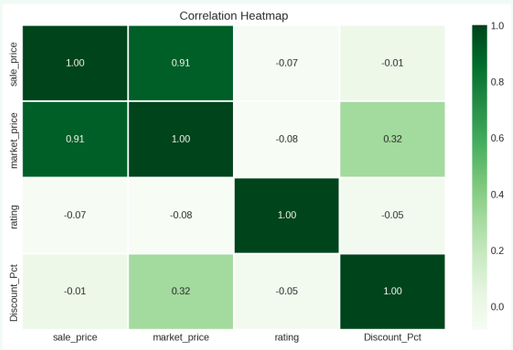
stack = pd.crosstab(df['category'], df['On_Discount'])
stack.plot(kind='barh', stacked=True, color=[GREEN, '#B3E6C4'], figsize=(9, 4.6))
plt.title('Discount Status across Categories')
plt.xlabel('Number of products')
plt.tight_layout()
plt.show()
```

What this code does?

- df.corr() calculates the correlation between every pair of numeric columns.
- sns.heatmap() draws it as a coloured grid; darker green means a stronger link.
- pd.crosstab() counts discounted and full-price products per category for the stacked bar.

2 Output Screenshot

Out *Output chart: correlation heatmap*



Out *Output chart: stacked bar*

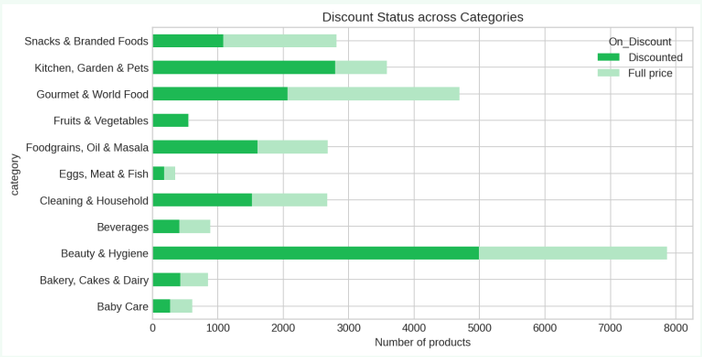


Figure 20 and 21: *Correlation heatmap of the numeric columns (left) and stacked bar chart of discount status across categories (right).*

What it shows?

- sale_price and market_price are strongly correlated (0.91).
- market_price and Discount_Pct have a moderate link (0.32): dearer items get bigger discount gaps.
- rating has very weak correlation with everything else (all |r| ≤ 0.08).
- Discounted share by category: highest Fruits & Vegetables 97.5%, lowest Snacks & Branded Foods 38.4%.

Key Insight

Only price and market price move together strongly; ratings do not depend on price or discount. Categories differ clearly in how often they are discounted, which is worth tracking.

✔ Correlations read	✔ Weak ratings link	✔ Category split seen	✔ Big picture clear

Many Variables → Big Picture

SPOTIFY | DATA ANALYTICS

CONCLUSION

Turning data into retail insights

The Spotify Project Analysis provided clear insights into the catalogue by applying **Python**, **Pandas**, **NumPy**, **Matplotlib** and **Seaborn** for data cleaning, analysis and visualisation.

★ Key Findings

1	**27,555 products, 10 columns** The dataset has no duplicates; 8,759 missing cells in 5 columns were filled with median, mode or a fixed text.
2	**Prices are right-skewed** Mean Rs. 335 vs median Rs. 190; 10,724 outliers across three columns were replaced with the mean.
3	**Ratings are incomplete** 31.3% of products (8,636) had no rating; typical rating is 4.1.
4	**Beauty & Hygiene leads** 7,867 products (28.6%) and the largest total sale value of the catalogue.
5	**Discounts are common** 57.6% of products are sold below market price; a 10% extra offer on Beauty & Hygiene would save Rs. 340,475.
6	**Price follows market price** Sale price vs market price is the strongest relationship (correlation 0.91).
7	**Ratings are unrelated to price** Rating correlates with other columns by at most 0.08.

Tools Used

Python | Pandas | NumPy | Matplotlib | Seaborn | Jupyter / Colab

Final Takeaway

Data turns a raw product catalogue into clear, measurable insights that support better retail decisions.

Limits of this analysis: the file has no sales or quantity column, so true best-sellers cannot be measured (price and listing counts were used instead); Task 3 (maps) was skipped because there is no geographic data. Mean replacement of outliers keeps all rows but flattens real price differences.

Thank You! ♥

SPOTIFY • BIGBASKET DATASET • DATA ANALYTICS



THANK YOU

Data turns a raw product catalogue into clear, measurable insights that support better retail decisions.

PYTHON • PANDAS • NUMPY • MATPLOTLIB • SEABORN

Better Data → Better Decisions → Smarter Retail